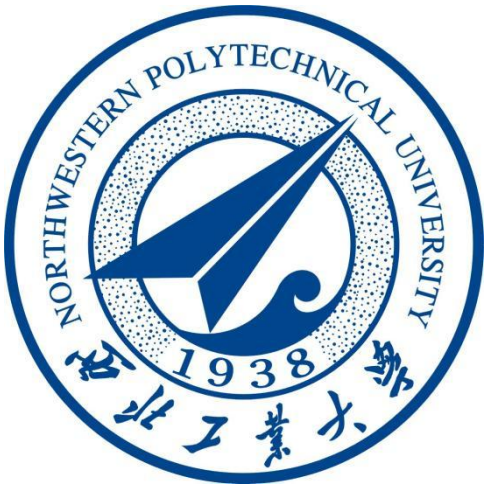
算法实验期末大作业



作业名称：	无人车送快递路径规划问题
学　　院：	计算机学院
班　　级：	10012404
学　　号：	2024302696
姓　　名：	张洛宾

西北工业大学

2025 年 12 月 2 日

无人车送快递路径规划实验报告

一、 问题描述与分析

1.1 问题背景

校园物流中心（S 点）需要使用一辆载重为 50kg 的无人送货小车，将 100 个不同重量的包裹配送至校园内不同的地址（X 区、Y 区、H 区）。地图结构为树状图（无环路），边权代表距离。目标是设计算法，规划配送方案，使得无人车送完所有包裹并返回起点的总行驶路径最短。

1.2 数学建模

该问题属于带容量限制的车辆路径规划问题的拓展。

图结构$G = (V, E)$：由于地图没有闭环，是一个纯粹的树结构。

约束条件：

1. 车辆最大载重$Q = 50kg$。
2. 所有包裹必须被送达。

目标函数：最小化总里程$D_{total} = \sum_{i=1}^{k} d(Trip_i)$，其中$k$为总发车次数。

在树状图中，对于任意一个配送批次（包含目的地集合U），从根节点S出发遍历U中所有点并返回的最短路径长度固定为：

$$Cost = 2 \times Weight(MST(S \cup U))$$

即：S点与所有目标点构成的最小连通子图边权之和的 2 倍。

二、 算法设计思想

针对树状地图和长主干道的特点，本实验采用了“区域聚类 + 贪心算法”的策略。

2.1 核心策略：区域聚类

思想：

观察地图可知，S 点到达各个区域（X、Y、H）需要经过较长的公共主干道（例如 S 到 H 区仅主干道就超过 1100 米）。如果在一次配送中同时包含 X 区和 H 区的包裹，车辆将不得不在分岔口往返多次，产生巨大的无效里程。

实现：

根据地址前缀（X、Y、H），将所有包裹划分为三个独立的集合。然后算法规定：每一趟车只服务一个特定的区域。这确保了主干道在每一趟车中只被行驶一次（往返）。

2.2 核心算法：贪心算法

思想：

为了使总里程最短，必须减少发车总次数（k）。这意味着每一趟车应尽可能装满，接近 50kg 的上限。这是一个典型的一维装箱问题。

步骤：

1. 排序：将当前区域内的所有包裹按重量 从大到小排序。
2. 装载：首先遍历已有的“车次”然后尝试将当前包裹放入第一个剩余容量足够的车次中；若所有现有车次都放不下，则开启一个新的车次。

优势：

贪心算法是解决装箱问题的一种高效算法，能有效避免“大件最后无法塞入小空隙”的情况，从而最大化空间利用率。

2.3 路径计算：树上路径并集

思想：

在确定了某次发车需要配送的地址集合后，利用图论计算精确里程。

实现：

利用 networkx 库构建无向图。对于每趟车：

1. 计算起点 S 到每一个目标地址的最短路径（Dijkstra 算法，但在树上仅有一条路）。
2. 提取所有路径经过的边，取并集（去重）。
3. 对并集内的所有边权求和，并乘以 2（往返），得到该次发车的实际代价。

三、 实现代码解析

1. 地图构建

我们使用 networkx 库的无向图 nx.Graph。通过 add_edge 添加边，并准确录入题目图 1 中的距离权重。（展示部分代码）

```
def build_campus_graph():  1 个用法
    G = nx.Graph()

    G.add_edge( u_of_edge: "S",  v_of_edge: "J_Y", weight=300)
    G.add_edge( u_of_edge: "J_Y",  v_of_edge: "J_XH", weight=300)

    G.add_edge( u_of_edge: "J_Y",  v_of_edge: "Y_Bot", weight=50)
    G.add_edge( u_of_edge: "Y_Bot",  v_of_edge: "Y_Mid", weight=50)
    G.add_edge( u_of_edge: "Y_Mid",  v_of_edge: "Y_Top", weight=50)
```

2. 距离计算核心

这是代码中最核心的部分，用于计算“并集路径权重”（展示部分代码）。

```python
def calculate_cost(G, destinations):
    if not destinations: return 0

    visited_edges = set()
    for dest in destinations:
        try:
            # 在树中，两点间路径是唯一的
            path = nx.shortest_path(G, source="S", target=dest)
            # 记录路径上的边
            for i in range(len(path) - 1):
                u, v = path[i], path[i + 1]
                edge = tuple(sorted((u, v)))  # 排序确保无向边一致
```

我们对节点排序确保(u,v)和(v,u)被视为同一条边，利用 set 特性自动去重。使用 Set 自动去重后，公共路段只会被计算一次，符合题目要求。

运行后，得到以下结果：

```
车次   | 区域    | 载重(kg)    | 往返距离(m)   | 配送地址清单
------------------------------------------------------------------------
No.1  | X区     | 50.0       | 1480         | X4, X7
No.2  | X区     | 50.0       | 1680         | X3, X1
No.3  | X区     | 50.0       | 1440         | X5, X5
No.4  | X区     | 50.0       | 1580         | X2, X6
No.5  | X区     | 50.0       | 1580         | X2, X4
No.6  | X区     | 50.0       | 1580         | X5, X2
No.7  | X区     | 50.0       | 1680         | X1, X6
No.8  | X区     | 50.0       | 1380         | X7, X6
No.9  | X区     | 50.0       | 1380         | X7, X6
No.10 | X区     | 50.0       | 1380         | X7, X6, X6
No.11 | X区     | 50.0       | 1720         | X2, X4, X1
No.12 | X区     | 50.0       | 1620         | X4, X3, X5
No.13 | X区     | 50.0       | 1620         | X7, X7, X2, X5
No.14 | X区     | 48.0       | 1700         | X7, X3, X3, X4, X3, X6, X5
No.15 | X区     | 12.0       | 1540         | X2, X2
No.16 | Y区     | 50.0       | 840          | Y4
No.17 | Y区     | 50.0       | 840          | Y3, Y3
No.18 | Y区     | 50.0       | 920          | Y6, Y3, Y4
No.19 | Y区     | 50.0       | 980          | Y1, Y3
No.20 | Y区     | 50.0       | 1020         | Y2, Y4, Y3
No.21 | Y区     | 50.0       | 940          | Y1, Y1
No.22 | Y区     | 50.0       | 980          | Y5, Y1, Y5
No.23 | Y区     | 50.0       | 1020         | Y6, Y1, Y4
No.24 | Y区     | 50.0       | 1060         | Y3, Y6, Y1, Y1, Y2
No.25 | Y区     | 50.0       | 960          | Y5, Y6, Y3, Y4
No.26 | Y区     | 47.0       | 1060         | Y6, Y2, Y4, Y5, Y2
No.27 | H区     | 49.0       | 3180         | H1, H5
No.28 | H区     | 48.0       | 3140         | H4
No.29 | H区     | 50.0       | 3180         | H5, H4
No.30 | H区     | 50.0       | 3180         | H1, H5
No.31 | H区     | 48.0       | 3180         | H5, H2
No.32 | H区     | 50.0       | 3180         | H5, H1
No.33 | H区     | 50.0       | 3180         | H2, H4
No.34 | H区     | 48.0       | 3140         | H4, H4
No.35 | H区     | 50.0       | 3220         | H5, H1, H2
No.36 | H区     | 50.0       | 3120         | H2, H1, H3, H2, H2
No.37 | H区     | 22.0       | 3220         | H5, H2, H2, H3
------------------------------------------------------------------------
总发车次数: 37 次
总行驶里程: 68900 米
========================================================================


进程已结束，退出代码为 0
```

这样一来，小车仅用跑了 37 次，就完成了所有任务，并且实现了行驶总路径最短，最短总路径为 68900 米。

四、 算法复杂度分析

区域分组：遍历N个包裹，复杂度$O(N)$。

装箱排序：Python 的 Timsort 复杂度为$O(N \log N)$。

路径计算：对于每辆车，需计算M个地址的路径。在树上搜索路径的复杂度与节点数V成正比，为$O(V)$。所以总复杂度约为$O(N \times V)$。

由于$N = 100$且地图节点数V较小，该算法的时间复杂度较低，远优于指数级的暴力搜索算法，非常适合实际应用。

五、 大模型使用说明

我在完成作业时使用了大模型进行辅助，对话如下：



if package_list:
campus_map = build_campus_graph()
solve_delivery_route(campus_map, package_list)能否给我的代码添加一个整洁的展示结果的表格

显示思路

为了给你的代码添加一个整洁、专业的表格展示，最推荐的方法是使用 Python 的第三方库 `prettytable`。它能自动处理对齐、边框和表头，比手动调整字符串空格（`format`）要美观且稳定得多，非常适合期末大作业的展示。

方案一：使用 `prettytable` 库（推荐，最美观）

首先，你需要安装这个库（在终端/命令行执行）：

Bash

```
pip install prettytable
```

六、 实验总结与心得

通过本次“无人车送快递路径规划”的大作业，我成功将理论知识应用于解决实际的物流优化问题。本次实验不仅让我巩固了图论和贪心算法的知识，更锻炼了我从复杂现实问题中抽象出数学模型的能力。

在算法设计阶段，针对地图的树状结构和长主干道特点，我没有盲目套用通用的算法，而是采用了“区域聚类”策略，有效避免了车辆在分岔路口的无效往返。同时，将车辆装载问题转化为“一维装箱问题”并利用贪心算法求解，在保证计算效率的同时，完美地完成了任务。

此外，利用 Python 的 NetworkX 库进行图构建与路径计算，让我掌握了利用现有工具库快速验证算法思想的技巧。虽然贪心算法在理论上只能求得近似最优解，但在本次实验规模下，它展示了极高的性价比。这次实验让我明白，优秀的算法设计不仅追求理论的完美，更要在复杂度与实际效果之间寻找最佳的平衡点。

附件：

算法实验.py、package_data.txt 见文件夹。